



```
In [ ]: import torch
import pandas as pd
from transformers import BertTokenizer, BertForSequenceClassification
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
```

```
In [ ]: file_path = "/kaggle/input/datasets/ishant263/tweet/Tweets (1).csv"

df = pd.read_csv(file_path)

df.head()
```

```
In [ ]: texts = df['selected_text'].astype(str).tolist()

labels_raw = df['sentiment'].tolist()

label_map = {label: idx for idx, label in enumerate(sorted(set(labels_raw)))}
labels = [label_map[label] for label in labels_raw]

print("Label Mapping:", label_map)
```

```
In [ ]: tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

encodings = tokenizer(
    texts,
    padding=True,
    truncation=True,
    return_tensors="pt"
)
```

```
In [ ]: class SimpleDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = torch.tensor(labels)

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = self.labels[idx]
        return item

    def __len__(self):
        return len(self.labels)

dataset = SimpleDataset(encodings, labels)
loader = DataLoader(dataset, batch_size=8, shuffle=True)
```

```
In [ ]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
```

```

        num_labels=len(label_map),
        output_attentions=True
    )

    model.to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

```

```

In [ ]: model.train()

for epoch in range(2):
    print(f"\nEpoch {epoch+1}")

    total_loss = 0

    loop = tqdm(loader, leave=True)

    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}

        optimizer.zero_grad()

        outputs = model(**batch)
        loss = outputs.loss

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        loop.set_postfix(loss=loss.item())

    avg_loss = total_loss / len(loader)
    print(f"Average Loss: {avg_loss:.4f}")

```

```

In [ ]: model.eval()

sample_text = texts[0]

inputs = tokenizer(sample_text, return_tensors="pt").to(device)
outputs = model(**inputs)

attentions = outputs.attentions

tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])

attention = attentions[-1][0][0].detach().cpu().numpy()

plt.figure(figsize=(10, 8))
sns.heatmap(attention, xticklabels=tokens, yticklabels=tokens, cmap="viridis",
            fmt=".2f", annot_kws={"size": 8})
plt.title("Attention Heatmap (Last Layer, Head 0)")
plt.xlabel("Key Tokens")

```

```
plt.ylabel("Query Tokens")
plt.xticks(rotation=90)
plt.show()
```

```
In [ ]: from sklearn.metrics import accuracy_score

model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for batch in loader:
        batch = {k: v.to(device) for k, v in batch.items()}

        outputs = model(**batch)
        preds = torch.argmax(outputs.logits, dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(batch['labels'].cpu().numpy())

print("Accuracy:", accuracy_score(all_labels, all_preds))
```

```
In [ ]:
```

```
In [ ]:
```